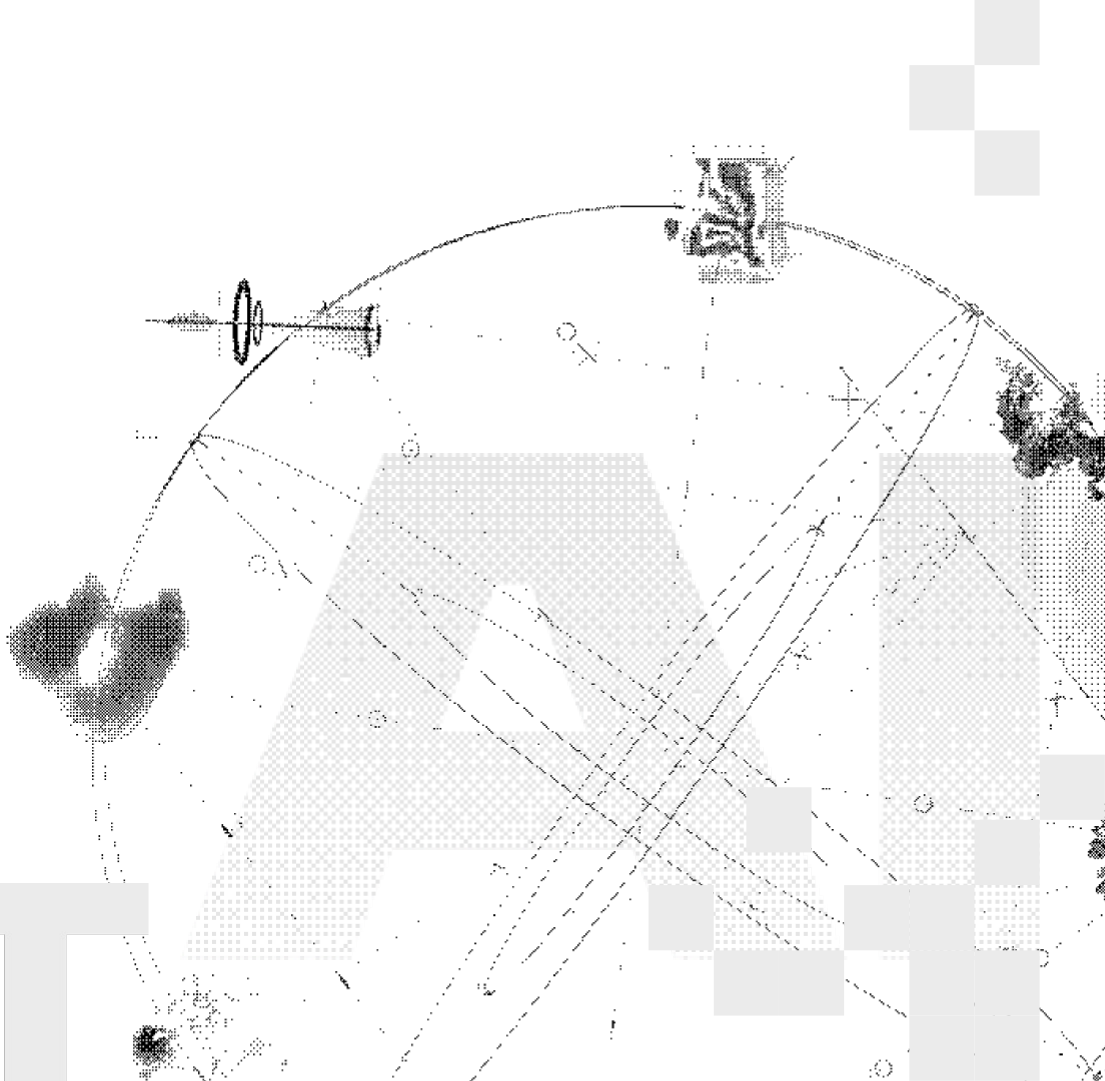


Python 프로그래밍

: Level 1 · 5강

HANT



이 시간의 목표

venv 로 프로젝트별 패키지를 **격리**하고 Jupyter 로 탐색·기록한다

list·dict·set·tuple 을 시간 복잡도 기준으로 선택한다

클래스 **상속·다형성**과 **타입 힌트**로 코드를 구조화한다

ROS2 노드, 인지 알고리즘, 시뮬레이션 — Python 은 로봇 개발 어디에나 등장합니다.
문법 나열이 아니라, 로봇 코드를 구조적으로 쓰기 위한 뼈대를 다룹니다.

목차



순서	주제	예상 소요 시간
1부	가상환경과 Jupyter — 실습 환경 구축	20분
2부	자료구조 선택과 함수·모듈·패키지	35분
3부	객체지향 설계와 타입 힌트	35분

1부

가상환경과 Jupyter

프로젝트마다 다른 패키지 버전이 필요합니다

① 혼한 사고

venv 가 없을 때

시스템 Python 에 계속
설치하다 버전이 충돌하고
OS·ROS2 도구까지 깨집니다

“분명 설치했는데 안 돼요”

venv 가 있을 때

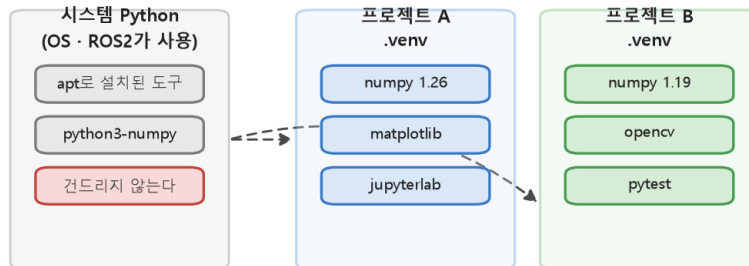
프로젝트마다 독립된
패키지 공간을 만들고
지우고 다시 만들면 그만입니다

“시스템 Python 은 성역”

↳ venv 는 프로젝트마다 독립된 패키지 공간을 만드는 도구입니다.

시스템 Python 은 성역, 작업은 항상 venv 안에서

가상환경 — 프로젝트마다 독립된 패키지 공간



같은 numpy라도 버전이 다른 두 방이 충돌 없이 공존

시스템 Python은 성역 — 모든 작업은 프로젝트별 .venv 안에서

해석

프로젝트마다 자기만의 패키지 방을 만들고, 방 하나가 꼬여도 지우고 다시 만들면 됩니다

pip freeze > requirements.txt 로 설치 목록을 기록해 팀과 공유합니다.

“내 컴퓨터에선 되는데요”의 상당수가 이 재현 절차 하나로 예방됩니다.

명령은 다섯 줄이면 충분합니다

생성·활성화	<code>python3 -m venv .venv</code> — 프로젝트 디렉터리에서 1회 <code>source .venv/bin/activate</code> — 프롬프트에 <code>(.venv)</code> 표시
설치·기록	<code>pip install numpy matplotlib</code> — 이 방에만 설치됩니다 <code>pip freeze > requirements.txt</code> — 설치 목록을 기록
재현·해제	<code>pip install -r requirements.txt</code> — 팀원이 같은 환경 재현 <code>deactivate</code> — 방에서 나오기

이 강의의 포인트

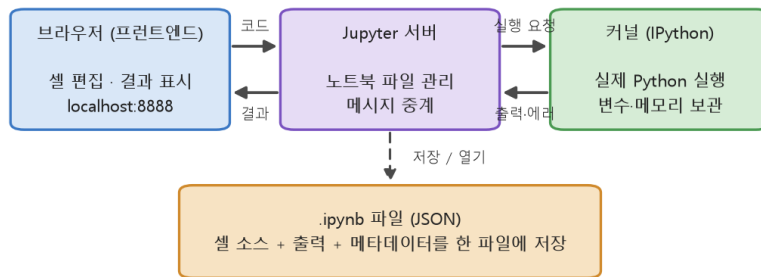
재현 절차 하나가 팀 전체의 시간을 지킵니다

“내 컴퓨터에선 되는데요”의 상당수가 `requirements.txt` 로 예방됩니다.

이 가상환경은 모듈 ④ 미니 프로젝트까지 그대로 이어집니다.

Jupyter — 셀 단위로 탐색하고 기록하는 도구

Jupyter Notebook의 3층 구조 — 브라우저 · 서버 · 커널



코드는 브라우저에서 쓰지만 실행은 커널이 한다 — 커널이 죽으면 변수도 사라진다

해석

화면은 브라우저일 뿐, 코드는 Jupyter 서버를 거쳐 커널에서 실행됩니다

변수와 import 는 전부 커널의 메모리에 있습니다.

ipykernel install 을 빠뜨리면 시스템 파이썬으로 돌아 ImportError 가 납니다.

셀은 독립적이지 않습니다 — 같은 커널 메모리를 잇는 조각

셀 실행 순서 — 노트북에서 가장 흔한 함정



실행 번호 [n]이 위에서 아래로 증가하지 않으면 결과를 얻을 수 없다 — 제출 전 Restart & Run All

해석

화면의 코드 순서와 실제 실행 이력은 다를 수 있습니다

실행 카운터 [n] 이 위에서 아래로 증가하는지가 판별 기준입니다.

제출·공유 전 Restart Kernel and Run All Cells 로 반드시 확인합니다.

다섯 개면 충분합니다



자주 쓰는 단축키

- **Shift + Enter** — 셀 실행 후 다음 셀로 이동
- **Ctrl + Enter** — 셀 실행 후 그 자리에 머무름
- **Esc → A / B** — 위 / 아래에 새 셀 삽입
- **Esc → M / Y** — 마크다운 셀 / 코드 셀로 전환
- **Esc → D D** — 셀 삭제

노트북은 탐색에 강하고, 재사용엔 약합니다.



습관 두 가지

- **Restart & Run All** — 제출·공유 전 위에서 아래까지 한 번에 통과하는지 확인
- **sys.executable 확인** — `import sys; print(sys.executable)` 로 커널 경로를 항상 점검
- 두 번 쓰는 코드는 .py 모듈로 옮깁니다 — 3절에서 기준을 다룹니다
- **.ipynb 파일** 은 셀의 소스·출력을 담은 JSON — Git 버전 관리도 됩니다(4강)
- 6강 NumPy 부터 모듈 ③·④ 실습까지 **이 환경을 그대로 이어 씁니다**

2부

자료구조와 코드 구조화

네 가지 내장 자료구조, 각각 다른 문제를 푼다

네 가지 내장 자료구조 — 문제에 맞게 고르기



포함 검사 `x in a` — list는 O(n) 전부 뒤집 · dict/set은 O(1) 해시로 즉시

30Hz 루프 안의 list 'in' 검사는 데이터가 쌓일수록 느려지는 성능 버그가 된다

해석

순서 있는 목록은 **list**, 불변 묶음은 **tuple**, 이름 조회는 **dict**, 소속 검사는 **set**

dict·set 의 조회는 O(1) — 데이터가 백만 개여도 한 번에 찾습니다.

“다 list 로” 짜는 코드와 알맞게 고른 코드는 가독성·성능이 크게 다릅니다.

로봇 코드에서 언제 무엇을 쓸까요

CASE 01

LiDAR 거리 같은 시계열, 계속 추가

list

append() 로 누적

CASE 02

홈 좌표처럼 바뀌면 안 되는 묶음

tuple

불변 — 실수 방지

CASE 03

센서 이름으로 값을 즉시 찾는다

dict

O(1) 조회

CASE 04

중복 없이 소속만 빠르게 검사

set

O(1) in 검사

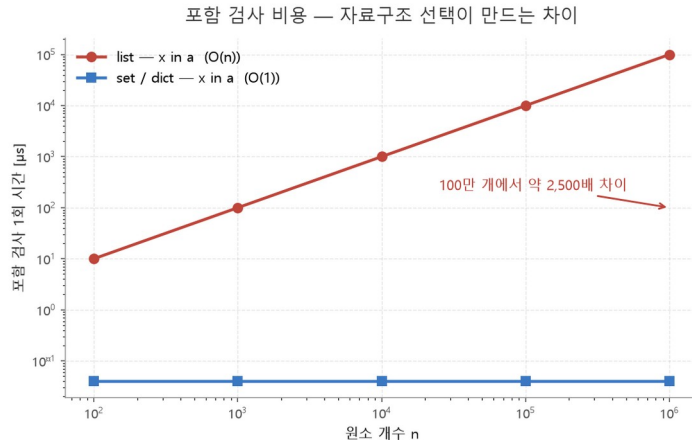
CASE 05

list 로 in 검사를 반복한다

성능 버그

O(n)×N, 지터 유발

포함 검사 $O(n)$ vs $O(1)$ — 얼마나 벌어질까요



데이터가 커질수록 벌어진다 — 루프 안이라면 곧바로 마감(deadline) 위반으로 이어진다

해석

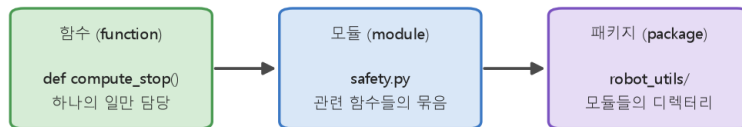
100만 개에서 list 는 100만 번 뒤지고 dict/set 은 한 번에 찾습니다 — 약 2,500배

30Hz 인지 루프의 list in 검사는 처음엔 멀쩡하다 마감을 놓치는 성능 버그가 됩니다.

2강의 주기·지터 문제와 직결됩니다. %timeit 으로 노트북에서 직접 재 봅니다.

코드는 함수 → 모듈 → 패키지 순서로 묶어 올립니다

코드 구조화 — 함수 → 모듈 → 패키지



```

robot_utils/           ← 패키지 (디렉터리)
├─ __init__.py         ← 패키지임을 알리는 표식
├─ kinematics.py       ← 모듈: 좌표·속도 계산
└─ safety.py           ← 모듈: 정지거리·한계 검사

from robot_utils.safety import compute_stop_distance
  
```

ROS2의 Python 패키지도 정확히 이 구조 위에 서 있다

해석

하나의 일을 하는 함수가 모여 모듈이, 관련 모듈이 모여 패키지가 됩니다

두 번 이상 쓰이기 시작하면 노트북에서 .py 모듈로 옮깁니다.

ROS2의 Python 패키지(모듈 ②)가 정확히 이 구조 위에 서 있습니다.

함수는 동사구로 짓습니다.

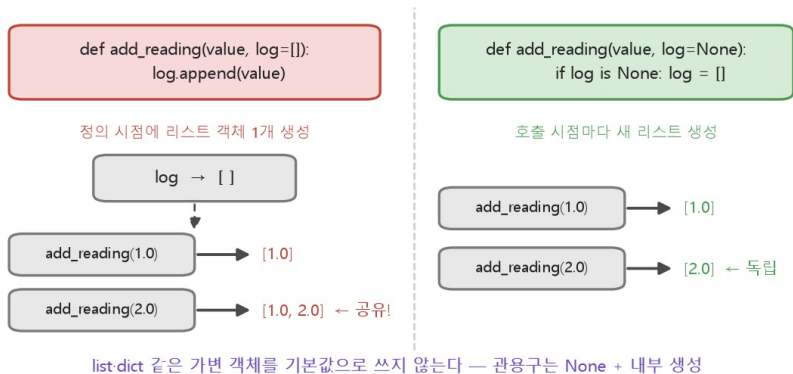


좋은 함수의 조건

- 이름은 동사구 — `compute_stop_distance` 처럼 하는 일이 이름에 드러나게
- 하나의 함수는 하나의 일만 — 여러 일을 섞으면 재사용도 테스트도 어려워집니다
- **docstring** — 무엇을 받고 무엇을 주는지, 함수의 계약을 설명
- **robot_utils/** 패키지 — `kinematics.py`, `safety.py` 처럼 역할별 모듈로
- ROS2의 Python 패키지(모듈 ②)가 **정확히 이 구조 위에 섭니다**

심화 — 가변 기본 인자, Python 의 유명한 함정

가변 기본 인자 — 기본값은 함수 정의 시점에 단 한 번 만들어진다



해석

기본값 [] 은 함수 정의 시점에 단 한 번 만들어져 모든 호출이 공유합니다

log=[] 대신 log=None 으로 두고, 함수 안에서 새로 만드는 것이 올바른 관용구입니다.

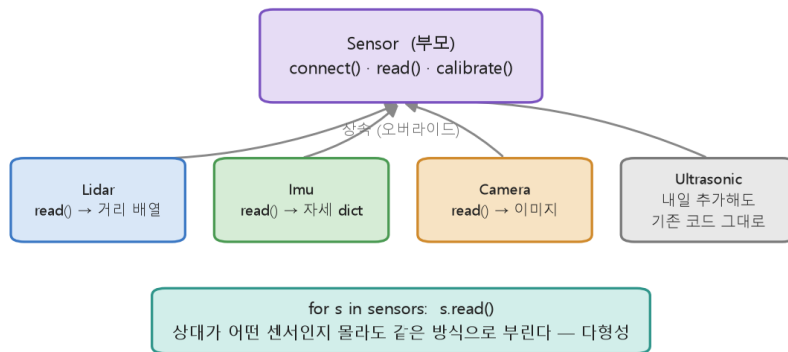
b = a 도 복사가 아니라 같은 객체에 이름 하나를 더 붙이는 것 — 필요하면 a.copy().

3부

객체지향과 타입 힌트

공통 동작은 부모에, 각자의 방식은 자식에

상속과 다형성 — 공통 동작을 추상화하면 확장이 공짜가 된다



ROS2의 class `MyNode(Node)` 도, 시뮬레이션 플랜트 모델도 모두 이 구조

해석

Sensor 를 상속한 **Lidar·Imu** 가 **read()** 를 각자 오버라이드합니다

`for s in sensors: s.read()` — 어떤 센서인지 몰라도 같은 방식으로 부릅니다.

이것이 다형성입니다. 새 센서가 추가돼도 기존 루프는 한 줄도 바뀌지 않습니다.

로봇 소프트웨어는 이 구조 위에 섭니다.



상속·다형성이 쓰이는 곳

- **Sensor** 부모 — connect·read 공통 인터페이스, read 는 자식이 반드시 구현
- **Lidar·Imu** 자식 — 같은 read() 이름, 반환 타입은 각자 다르게(오버라이드)
- **sensors: list[Sensor]** — 부모 타입으로 묶어 순회, 실제 타입은 몰라도 됩니다
- **확장** — 초음파 센서가 추가돼도 상속 클래스 하나면 충분
- ROS2의 Node 클래스(모듈 ②)도 **정확히 같은 구조입니다**

__이름__ 으로 파이썬답게 만듭니다.

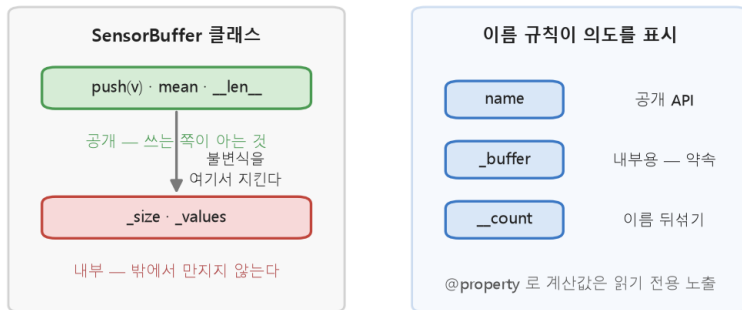


매직 메서드 다섯 개

- `__init__` — 생성. `Vector2(3.0, 4.0)` 처럼 만들 때 호출
- `__repr__` — 표현. `print(v)` 가 읽기 좋은 문자열을 보여줍니다
- `__add__` — 덧셈. `v1 + v2` 를 정의한 대로 계산합니다
- `__eq__` · `__len__` — 비교(==)와 길이(`len()`)를 자연스럽게
- 좌표·속도 벡터를 이렇게 감싸면 **수식이 코드에 그대로 보입니다**

공개할 동작과 감출 상태를 나눕니다

캡슐화 — 공개할 동작과 감출 상태



상태는 감추고 동작만 공개 — 내부를 바꿔도 쓰는 코드가 깨지지 않는다

해석

밖에 보이는 것은 `push·mean·len()` 뿐, `_size·_values` 는 안쪽에 둡니다

상태를 감추면 “길이는 size 를 넘지 않는다” 같은 불변식을 스스로 지킵니다.

`@property` 로 감싼 `mean` 은 속성처럼 읽히지만 대입은 막습니다.

파이썬은 이름 규칙으로 의도를 표시합니다

접근을 강제로 막지 않는 대신, 밑줄 개수로 의도를 알립니다.

`name`

공개 — 밖에서 써도 됨

`sensor.name` 처럼 그대로 노출

`_buffer`

내부용 — 건드리지 말 것

“밖에서 만지지 마세요”라는 약속

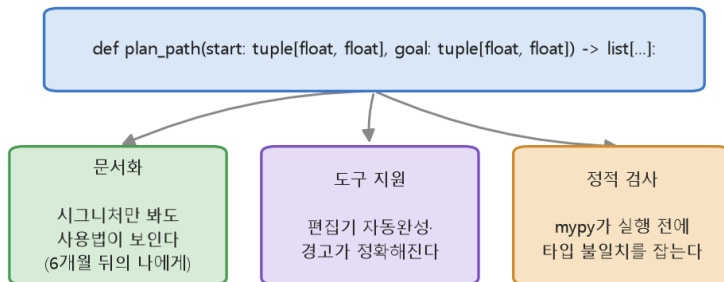
`__count`

이름 뒤섞기

상속 클래스와 이름이 겹치는 것도 방지

힌트 한 줄이 세 방향으로 동시에 일합니다

타입 힌트 — 사람과 도구를 위한 계약서



주의 — 인터프리터는 힌트를 무시하고 실행한다. 강제하려면 CI에 mypy를 넣는다.

로봇에서는 "실행해 보니 죽더라"의 비용이 물리적 사고 — 실행 전 검증의 가치가 크다

해석

문서화 · 도구 지원 · 정적 검사 — mypy 가 실행 전에 타입 불일치를 잡습니다

로봇에서는 “실행해 보니 죽더라”의 비용이 물리적 사고입니다.

타입 힌트는 강제가 아닙니다 — mypy 를 CI(4강)에 넣어야 강제됩니다.

심화 — dataclass



필드 몇 개를 한 줄로

- **@dataclass** — `__init__` · `__repr__` · `__eq__` 를 자동 생성
- 타입 힌트가 필드 정의 역할을 하는, “힌트가 실제로 일하는” 사례
- **RobotState(x, y, heading, battery=100.0)** 처럼 기본값도 그대로
- 로봇 상태·설정값 객체에 즐겨 쓰입니다 — 모듈 ④에서 다시 만납니다
- 일반 클래스보다 **타이핑이 훨씬 짧아집니다**

오늘 정리



이것만 남기면 됩니다

- **환경** — venv 로 패키지를 격리하고, Jupyter 는 브라우저·서버·커널 3층 구조
- **자료구조** — list·tuple·dict·set, 조회가 맞으면 $O(1)$ 인 dict/set 을 고른다
- **구조화** — 함수 → 모듈 → 패키지, 가변 기본 인자 함정을 피한다
- **객체지향** — 상속·다형성으로 확장에 열고, 매직 메서드로 파이썬답게
- 타입 힌트는 사람과 도구를 위한 **계약서입니다**

미니 퀴즈 (각 2점 · 총 10점)



스스로 확인하기

- 1. venv 를 쓰는 가장 큰 이유는?

- 2. 노트북에서 import 가 ImportError 로 실패할 때 가장 먼저 확인할 것은?

- 3. log=[] 를 기본 인자로 쓰면 안 되는 이유를 설명하세요.

- 4. 30Hz 루프에서 list 대신 set 을 써야 하는 이유는?

- 5. 타입 힌트가 실행 시점에 강제되는지 참/거짓으로 답하세요.

오늘의 실습

“센서 모니터링 패키지를 만들며
venv·자료구조·클래스·타입 힌트를
하나의 실습으로 잇습니다.”

Step 01

환경과 자료구조

venv·Jupyter 로 커널을 확인하고
dict·list·set·tuple 로 센서값을 다룹니다.

Step 02

클래스 계층

Sensor 를 상속해 Ultrasonic 을 추가하고
다형성 루프와 매직 메서드를 확인합니다.

Step 03

모듈화와 검사

%%writefile 로 sensors.py 로 내보내고
타입 힌트 후 mypy 로 검사를 통과시킵니다.

더 읽을 거리



참고 문헌

- **Python 공식 문서** (docs.python.org) — 표준 라이브러리와 언어 레퍼런스의 1차 자료
- **PEP 484** — 타입 힌트의 원 제안서. typing 모듈의 근거
- **Real Python** (realpython.com) — OOP·타입힌트·venv 실전 튜토리얼
- **Jupyter 공식 문서** (jupyter.org) — 커널·확장·단축키 전체 목록
- **mypy 공식 문서** (mypy-lang.org) — 정적 타입 검사 설정과 규칙

Q&A

무엇이든 물어보세요

환경부터 타입 힌트까지 —

다음 시간은 6강 NumPy 벡터 연산입니다